AUTHOR - Mohitha G

I am a Software Engineer with experience in building backend systems, data engineering solutions, and AI-based applications. I have worked on technologies like Python, SQL, FastAPI, and cloud platforms such as AWS and GCP.

This project demonstrates my understanding of backend development, including API design, database integration, and structured code organization.

I focus on writing clean, maintainable code and building scalable solutions aligned with real-world use cases.

FINANCE BACKEND API

1. PROJECT OVERVIEW

- This project is a backend application developed using Python (FastAPI) and SQLite.
- It is designed to manage users and their financial transactions, along with providing a simple dashboard to track income and expenses.

2. FEATURES IMPLEMENTED

- Create and view users
- Add and retrieve financial transactions
- Filter transactions by type (income/expense) and category
- Dashboard summary including:
  - Total income
  - Total expense
  - Net balance
  - Category-wise summary

3. TECHNOLOGY STACK

- Python
- FastAPI
- SQLite

4. PROJECT STRUCTURE

- main.py → Handles API routes

- database.py → Manages database connection and table creation
- models.py → Defines data structures
- services.py → Contains business logic

5. SETUP INSTRUCTIONS

- Install required packages: pip install fastapi uvicorn
- Run the application: uvicorn main:app --reload
- Open in browser: http://127.0.0.1:8000/docs

6. VALIDATION LOGIC

- User role is validated (admin, analyst, viewer)
- Transaction amount is ensured to be greater than zero

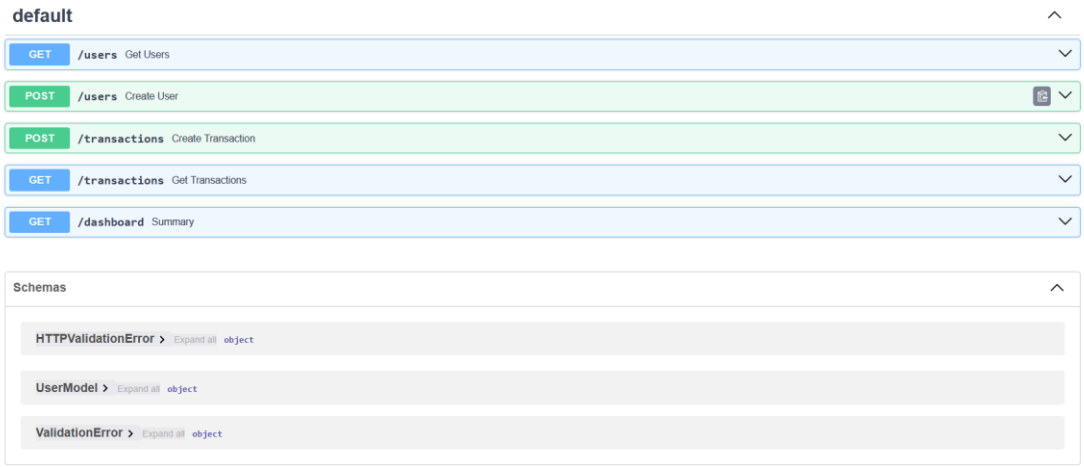
7. ADDITIONAL IMPLEMENTATION

- Added category-wise aggregation in the dashboard
- Implemented dynamic filtering for transactions
- Structured the code into separate layers for better clarity and maintainability

8. API TESTING & OUTPUT SCREENSHOTS

Below are the sample outputs from testing the APIs using Swagger UI.

1. Swagger UI Interface



2. Create User API

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/users' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "name": "Mohitha",
  "email": "mohitha@gmail.com",
  "role": "admin"
}'
```

Request URL

```
http://127.0.0.1:8000/users
```

Server response

Code	Details
200	Response body

```
{
  "message": "User created"
}
```

Download

Response headers

```
content-length: 26
content-type: application/json
date: Sun,05 Apr 2026 11:29:06 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

3. GET Users API

Request URL

```
http://127.0.0.1:8000/users
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "name": "Mohitha",
    "email": "mohitha@gmail.com",
    "role": "admin",
    "status": "active"
  }
]
```

Response headers

```
content-length: 88
content-type: application/json
date: Sun,05 Apr 2026 11:35:28 GMT
server: uvicorn
```

Responses

Code	Description
200	Successful Response

Media type

application/json

Controls Accept header.

4. Server Running in Terminal

```
C:\Users\mohit\OneDrive\Desktop\Finance_Backend Project>uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\mohit\\OneDrive\\Desktop\\Finance_Backend Project']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [10848] using StatReload
INFO:     Started server process [25304]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:64417 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:64417 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:64439 - "POST /users HTTP/1.1" 200 OK
INFO:     127.0.0.1:64440 - "GET /users HTTP/1.1" 200 OK
```

9. NOTES

- SQLite is used to keep the setup simple and lightweight
- Authentication is not added to keep the focus on core backend functionality